

RAP 後端開發練習 2：CDS Interface View 資料模型基礎 + Metadata Extension (UI Annotation) 入門

Lecture

Part A：資料模型——從 DDIC Table 到 CDS Interface View

rap01 講過 RAP 的五層架構，這一課動手蓋前兩層：**DDIC Table**（實際存資料）跟 **CDS Interface View**（資料模型，之後 rap03 的 Behavior Definition 會直接綁在這個 View 上）。

DDIC Table 設計要點（沿用 `.claude/rules/sap-adt-mcp.md` 第 34 / 39 節已經驗證過的 annotation 慣例）：

```
@EndUserText.label : 'RAP02 Task Root Table'
@AbapCatalog.enhancementCategory : #EXTENSIBLE_ANY
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #LIMITED
define table zrap02_task {
  key client : mandt not null;
  key task_id : zrap02_taskid not null;
  description : text100;
  status : zrap02_status;
  priority : zrap02_priority;
  due_date : zrap02_duedate;
  created_at : timestamp1;
  created_by : syuname;
}
```

兩個容易寫錯的地方（這次出題實際踩到、修正過的）：

- **timestamp1** 不能寫成 **abap.timestamp1**——CDS 的 **abap.*** 開頭是內建型別關鍵字（**abap.char** / **abap.dats** / **abap.int4** 這些），但 **TIMESTAMPL** 其實是一個標準 **Data Element**，要用 DDIC 物件引用的寫法（不加 **abap.** 前綴，直接寫 **timestamp1**，就像 **mandt** / **syuname** 一樣）。這兩種型別來源（CDS 內建 vs DDIC Data Element）語法看起來很像，容易搞混，寫錯的錯誤訊息是籠統的 **Can't save due to errors in source**，不會明確告訴你是這裡的問題。
- **created_at** / **created_by** 現在還用不到——這兩個欄位是先幫 rap05（Determination）鋪路的，讓 Managed BDEF 在建立資料時自動填入時間戳跟建立者，這一課先建好欄位，實際自動填值的邏輯留到 rap05。

Part A2：欄位型別一律要引用 Data Element，不要留 **abap.char()** 這種內建型別（硬性規則）

`.claude/rules/abap-style.md` 已經記錄了一條硬性規則：新建的 **DDIC** 物件，任何欄位如果在語意上對應到標準表的既有欄位（例如物料號碼對應 **MATNR**），一律要引用那個標準 **Data Element**，不可以自己用

`abap.char(...)` 重新定義一個長度剛好一樣的版本；如果真的是專案/課程獨有的業務概念、找不到對應的標準 **Data Element**，也不能偷懶留著內建型別，要自己建一組 **Domain + Data Element**。

這一課的五個非 Key 欄位剛好示範了兩種情況：

- ① 有現成標準 **Data Element** 可以直接重用——`description` 改用 `TEXT100` (`CHAR100`，套件 `BK`，語意中立、沒有綁定特定模組的通用文字欄位，用 ADT quickSearch 查證過，不是憑記憶猜的)。曾經考慮過 `priority` / `due_date` / `status` 有沒有現成標準 DE，quickSearch 查到的候選 (如 `/SAPAPO/PRIORITY`、`GM_TASK_STATUS`) 全部是特定模組專用 (APO、Grants Management)，硬套會誤導欄位語意，所以判定「沒有真正合適的標準 DE」，改走第二種情況。
- ② 沒有合適標準 DE，自己建 **Domain + Data Element**——`task_id` / `status` / `priority` / `due_date` 各自建了一組：

Domain	固定值	Data Element	標籤
ZRAP02_TASKID	無 (自由輸入 CHAR10)	ZRAP02_TASKID	Task ID
ZRAP02_STATUS	O=Open / D=Done	ZRAP02_STATUS	Status
ZRAP02_PRIORITY	H=High / M=Medium / L=Low	ZRAP02_PRIORITY	Priority
ZRAP02_DUEDATE	無 (DATS 日期)	ZRAP02_DUEDATE	Due Date

`status` / `priority` 這兩個特別示範了 Domain 固定值清單的建立語法 (`.claude/rules/sap-adt-mcp.md` 第 8 / 25 節記載的模式：`doma:fixValue` 只給 `doma:low` + 遞增 `doma:position`，不帶 `doma:high`，這種離散單值型的 `doma:text` 說明文字會正確存下來)。

⚠ 一個重要的觀念澄清：Domain 固定值清單只在畫面/UI 層級生效 (Fiori Elements 會自動把它變成 F4 下拉選單)，不是資料庫層的強制約束——程式用 `INSERT/MODIFY` 硬塞一個不在清單裡的值一樣會成功。也就是說，就算 `status` 已經有 Domain 固定值限制成只能是 `O/D`，`rap06` 的 **Validation** 邏輯還是必要的，Domain 給的是「使用者操作畫面的友善提示」，Validation 才是「真正擋掉非法資料」的防線。

⚠ **Data Element** 建立也踩到一個熟悉的坑：POST 建立 Data Element 時，即使 XML 裡帶了完整的 `shortFieldLabel` / `mediumFieldLabel` 等標籤，第一次 POST 回應雖然 201 成功，但標籤實際上沒有真的存進去 (跟 `.claude/rules/sap-adt-mcp.md` 第 8 節記載的「DE 標籤不會在 POST 當下落地」完全一致)——一定要再走一次 **LOCK→PUT** (同一份 XML) →**UNLOCK**，PUT 這一步才會真的把標籤寫進去，光靠 POST 不夠。

CDS Interface View：

```
@AbapCatalog.sqlViewName: 'ZIRAP02TASK'
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'RAP02 Task Interface View'
@ObjectModel.compositionRoot: true
@Metadata.allowExtensions: true
define root view ZI_RAP02_TASK
  as select from zrap02_task
{
```

```

key task_id,
description,
status,
priority,
due_date,
created_at,
created_by
}

```

四個必要 annotation，各自的作用：

Annotation	作用
<code>@AbapCatalog.preserveKey: true</code>	rap01 已經踩過的坑——這個系統的 RAP 根 View 缺了這行會直接啟用失敗 (" <code>@AbapCatalog.preserveKey: true</code> " missing for entity in BO structure)
<code>@ObjectModel.compositionRoot: true</code>	告訴 RAP 框架「這是一個獨立的 Business Object 根節點」，不是被其他 View 組合進去的子節點
<code>@AccessControl.authorizationCheck: #NOT_REQUIRED</code>	先關掉權限檢查，讓這一課專心在資料模型上；正式的 RAP 授權機制是進階主題，不在本課程範圍
<code>@Metadata.allowExtensions: true</code>	這一課新增的一行——沒開這個，Part B 的 Metadata Extension 會建不起來 (系統會說找不到可擴充的 View)

欄位別名與 `mapping for` 的取捨：注意 View 裡的欄位 (`task_id`、`description`.....) 沒有加 `as` 別名，直接沿用跟表格一樣的名稱。這是刻意的選擇——如果欄位改名 (例如 `task_id as TaskID`)，BDEF 那邊建立 Managed 行為時就需要另外寫一段 `mapping for zrap02_task { ... }` 明確對應每個欄位；欄位名稱維持一致，系統會自動對應，少一段要維護的程式碼。等以後真的有「View 欄位名稱要跟 OData 曝光的名稱不同」的需求時，才值得用別名 + mapping。

Part B：Metadata Extension——UI Annotation 語法入門

`.claude/rules/sap-adt-mcp.md` 第 40.10 節已經查證過：`@UI.*` Annotation 跟「Classic RAP / ABAP Cloud RAP」這條語法版本軸線無關，這個系統完全支援，只是外層包裹的語句關鍵字 (跟 CDS View 的 `define root view` vs `define root view entity` 一樣) 用的是舊式的 `annotate view`，不是新式的 `annotate entity`。

為什麼要獨立建一個物件，不直接寫在 **CDS View** 裡：`@UI.*` 這類 Annotation 純粹是「這個欄位在 Fiori Elements 畫面上怎麼顯示」，跟資料模型本身無關。拆到 Metadata Extension (DDLX，一個跟 **CDS View** 同名、但物件型別不同的獨立物件) 的好處是：核心資料模型維持乾淨，UI 客製化可以獨立維護、甚至以後讓不同團隊/國家疊加各自的畫面客製化而不用改到共用的 View 本體。

```

@Metadata.layer: #CUSTOMER
@UI: {
  headerInfo: {
    typeName: 'Task',
    typeNamePlural: 'Tasks',

```

```

    title: { type: #STANDARD, value: 'description' }
  }
}

annotate view ZI_RAP02_TASK with
{
  @UI.lineItem: [{ position: 10 }]
  @UI.selectionField: [{ position: 10 }]
  task_id;

  @UI.lineItem: [{ position: 20 }]
  description;

  @UI.lineItem: [{ position: 30 }]
  @UI.selectionField: [{ position: 20 }]
  status;

  @UI.lineItem: [{ position: 40 }]
  priority;

  @UI.lineItem: [{ position: 50 }]
  due_date;
}

```

三個基礎 **@UI.*** 標記，這一課只需要記住它們的角色分工：

- **@UI.headerInfo**：Entity 層級（寫在 **annotate view ... with** 外面），控制 Object Page（單筆詳細畫面）的標題怎麼顯示——這裡設定「顯示名稱是 Task/Tasks」「標題文字取 **description** 欄位的值」
- **@UI.lineItem**：欄位層級，標記這個欄位要不要出現在 List Report（清單）的表格欄位裡，**position** 決定欄位出現的順序
- **@UI.selectionField**：欄位層級，標記這個欄位要不要出現在清單畫面的篩選/搜尋條件裡

⚠ 這一課實測踩到的語法規則：**annotate view ... with { }** 區塊裡列出的每個欄位，**至少要有一個 Annotation**，不能空白列出——一開始把 **created_at / created_by** 也列進去但沒加任何標記，啟用時報 **Element 'created_at' must have at least one annotation**。修法很直接：這兩個欄位本來就不需要在畫面上曝光，直接不列進 **annotate view** 區塊即可（CDS View 裡沒被 Metadata Extension 提到的欄位，就是維持「沒有 UI 標記」的預設狀態，不會出錯）。

學習目標

- 能設計一張適合當 RAP 根實體的 DDIC Table（正確的 Key 結構、欄位型別選擇），分清楚 CDS 內建型別（**abap.***）跟引用 Data Element（不加前綴）兩種寫法的差異
- 知道欄位型別一律要引用 Data Element 的硬性規則：語意對應標準表既有欄位就直接重用標準 DE（先用 quickSearch 查證，不要憑記憶猜），真的沒有合適標準 DE 才自己建 Domain + DE，不可以留著 **abap.char(...)** 這類內建型別偷懶
- 能寫出 Domain 固定值清單的建立語法，並知道固定值清單只在 UI 層生效（F4 下拉），不是資料庫層的強制約束，跟 Validation（rap06）的角色不同

- 能寫出這個系統適用的 CDS Interface View 語法 (`define root view` , 四個必要 annotation 各自的作用)
- 理解欄位別名跟 `mapping for` 的取捨 , 知道什麼情況才需要用別名
- 能建立一個 Metadata Extension (DDLX) 物件 , 寫出這個系統適用的 `annotate view ... with { }` 語法
- 能寫出三個基礎 `@UI.*` 標記 (`headerInfo` / `lineItem` / `selectionField`) , 知道各自控制畫面的哪個部分
- 知道 Metadata Extension 裡列出的每個欄位都必須至少有一個 Annotation , 沒有 UI 需求的欄位乾脆不要列進去

物件清單

物件	名稱	型別
DDIC Table	ZRAP02_TASK	TABL/DT
Domain	ZRAP02_TASKID / ZRAP02_STATUS / ZRAP02_PRIORITY / ZRAP02_DUEDATE	DOMA/DD
Data Element	ZRAP02_TASKID / ZRAP02_STATUS / ZRAP02_PRIORITY / ZRAP02_DUEDATE	DTL/DE
CDS Interface View	ZI_RAP02_TASK	DDL/DF
Metadata Extension	ZI_RAP02_TASK (同名 , 型別不同)	DDLX/EX

全部物件都在 `$TMP` 套件 , 已建立並啟用 (`sap_inactive_objects` 確認 0 筆殘留) 。

驗證方式

Table / CDS View / Metadata Extension 都不是可執行的程式 , 沒有 `programrun` 這種無頭驗證手段 , 這一課的驗證重點是語法正確 + 成功啟用 :

1. 用 `checkruns` API (見規則檔第 4 節) 或直接嘗試啟用 , 確認沒有語法錯誤
2. `sap_inactive_objects` 回傳空清單 , 代表沒有殘留的未啟用版本
3. 讀回 `version=active` 的原始碼 , 逐字比對跟預期一致

真正「畫面長什麼樣子」的驗證 (Metadata Extension 的 `@UI.*` 標記實際效果) , 要等 rap04 建出 Service Binding、Publish 之後 , 用 Fiori Elements Preview 才看得到——這一課先確保語法正確、能啟用即可。

思考題

1. 如果之後想讓 `task_id` 由系統自動產生 (不用使用者手動輸入) , 資料型別跟目前的 `abap.char(10)` 手動輸入模式相比 , 通常會怎麼設計 ? (提示 : 想想 RAP 常見的 GUID / Early Numbering 概念 , 這是 rap03 會碰到的話題)
2. `@UI.selectionField` 只加在 `task_id` 跟 `status` 兩個欄位 , 如果之後想讓使用者也能用 `priority` 篩選 , 要怎麼修改 ?
3. 如果同一個 CDS View 之後想給兩種不同的畫面用 (例如一個給管理者看全部欄位、一個給一般使用者看精簡版) , Metadata Extension 「一個 View 只能對應一個」這件事 , 會不會是個限制 ? (提示 : 這就是

為什麼真實專案常常會另外設計 Projection View，让不同消費情境各自套用不同的 Metadata Extension——這個系統的單層模式簡化了這一步，多消費情境是進階主題)

答案

見 zrap02_task.tabl.abap、zrap02_taskid.doma.xml / .dtel.xml、zrap02_status.doma.xml / .dtel.xml、zrap02_priority.doma.xml / .dtel.xml、zrap02_duedate.doma.xml / .dtel.xml、zi_rap02_task.ddls.abap、zi_rap02_task.ddlx.abap。SAP 端物件：ZRAP02_TASK (Table)、四組同名 Domain / Data Element、ZI_RAP02_TASK (CDS View + Metadata Extension，同名不同型別)。